

1. El enlazado de programas e inicialización

Para convertir varios archivos fuente compilados en un único ejecutable funcional, el enlazador (*linker*) asigna direcciones físicas de memoria a cada instrucción y variable. Aunque para las instrucciones nos importa un poco menos, es importante asegurar que ciertos elementos de nuestro programa acaban en direcciones de memoria concretas. Por ejemplo, el código de arranque o ciertas funciones de interrupción (disparados ambos automáticamente) tendrán que estar en ubicaciones concretas.

1.1. El *linker script*

El *linker script* (archivo con extensión `.ld`) es el documento de configuración que indica al enlazador cómo organizar la memoria del sistema. Cumple tres funciones principales: define las regiones de memoria física (utilización de Flash y RAM), ubica las diferentes secciones de código (variables/funciones) en ellas, y además permite exportar símbolos de dirección para que el propio código sepa dónde acaba ubicado.

Un mapa de memoria típico de ARM Cortex-M divide las secciones del programa en:

- **.isr_vector:** Tabla de punteros con las llamadas a funciones de inicio e interrupciones.
- **.text:** Código ejecutable e instrucciones del programa (almacenado en Flash).
- **.rodata:** Constantes y cadenas de texto de solo lectura (almacenado en Flash).
- **.data:** Variables globales e estáticas *inicializadas* con un valor distinto de cero.
- **.bss:** Variables globales e estáticas *no inicializadas* (o inicializadas a cero).

```
1 MEMORY
2 {
3     RAM (xrw)  : ORIGIN = 0x20000000, LENGTH = 256K
4     FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 512K
5 }
6
7 SECTIONS
8 {
9     /* Tabla de vectores al inicio de la Flash */
10    .isr_vector : {
11        KEEP(*(.isr_vector))
12    } >FLASH
13
14    /* Código */
15    .text : {
16        *(.text*)
17        *(.rodata*)
18        _etext = .; //símbolo exportado al final del código
19    } >FLASH
20
21    /* .rodata (constantes, strings, etc.) */
22    .rodata :
23    {
24        *(.rodata)
25        *(.rodata*)
26    } >FLASH
```

```

27
28  /* Datos inicializados, dirección de comienzo en FLASH */
29  _sidata = LOADADDR(.data);
30
31  .data :
32  {
33      _sdata = .;          /* Inicio de datos en RAM */
34      *(.data)             /* .data sections */
35      *(.data*)            /* .data* sections */
36      _edata = .;         /* Fin de datos en RAM*/
37
38  } >RAM AT> FLASH /* Se guarda en RAM en ejecución, pero está en
39  FLASH en compilación */
40
41  /* Datos sin inicializar van a RAM directamente */
42  .bss : {
43      _sbss = .;
44      *(.bss*)
45      _ebss = .;
46  } > RAM
47
48  /* Definición del tope de la pila al final de la RAM */
49  _estack = ORIGIN(RAM) + LENGTH(RAM);
50 }

```

Es importante diferenciar la **LMA** (*Load Memory Address*) frente a **VMA** (*Virtual Memory Address*). La sección `.data` reside físicamente en Flash (**LMA**) para no perder sus valores al apagar el chip, pero el procesador debe ejecutarla y modificarla desde la RAM (**VMA**). Este tipo de inicialización la hace el código de inicio.

1.2. Código de inicio y salto a `main`

Cuando el microcontrolador recibe alimentación o se resetea, el procesador no ejecuta directamente la función `main()`. En su lugar, el hardware lee la primera dirección de memoria y salta automáticamente al `Reset_Handler`, definido en el archivo de ensamblador de arranque (`startup.s` o similar).

El `Reset_Handler` realiza la inicialización básica del sistema en el siguiente orden estricto:

1. **Inicialización del Stack Pointer (SP):** Carga la dirección del puntero de pila principal con el símbolo `_estack`.
2. **Llamada a `SystemInit`:** Configura los relojes principales (*clocks*) y la memoria Flash.
3. **Copia de la sección `.data`:** Lee el bloque de inicializadores desde la Flash (`_sidata`) y los escribe en la RAM (desde `_sdata` hasta `_edata`).
4. **Limpieza de la sección `.bss`:** Rellena con ceros la región de RAM comprendida entre `_sbss` y `_ebss`.
5. **Llamada a constructores globales (`__libc_init_array`):** Prepara librerías de C/C++.
6. **Salto a `main()`:** Salta a la rutina principal de la aplicación. Si esta finaliza, entra en un bucle infinito de seguridad (`LoopForever`).

```

1  Reset_Handler:
2      ldr    r0, =_estack
3      mov    sp, r0          /* set stack pointer */
4      /* Call the clock system initialization function.*/
5      bl     SystemInit
6
7      /* Copy the data segment initializers from flash to SRAM */
8      ldr r0, =_sdata
9      ldr r1, =_edata
10     ldr r2, =_sidata
11     movs r3, #0
12     b LoopCopyDataInit
13
14     /***** Sección de copia de datos inicializados *****/
15 CopyDataInit:
16     ldr r4, [r2, r3]
17     str r4, [r0, r3]
18     adds r3, r3, #4
19
20 LoopCopyDataInit:
21     adds r4, r0, r3
22     cmp r4, r1
23     bcc CopyDataInit
24
25     /***** Sección de inicialización con ceros de .bss *****/
26     ldr r2, =_sbss
27     ldr r4, =_ebss
28     movs r3, #0
29     b LoopFillZerobss
30
31 FillZerobss:
32     str r3, [r2]
33     adds r2, r2, #4
34
35 LoopFillZerobss:
36     cmp r2, r4
37     bcc FillZerobss
38
39     /***** Inicialización de librerías *****/
40     bl __libc_init_array
41
42     /***** Salto al programa principal *****/
43     bl main
44
45 LoopForever:
46     b LoopForever

```

1.3. La tabla de vectores

La arquitectura ARM Cortex-M requiere que las direcciones de las funciones de atención a excepciones e interrupciones estén organizadas en una posición fija de memoria, conocida como **Tabla de Vectores**. Por defecto, se ubica al principio de la memoria Flash (0x08000000).

Cada entrada de la tabla es una palabra de 32 bits que contiene la dirección de memoria de la rutina correspondiente:

- **Offset 0x00:** Puntero inicial de la pila (`_estack`).

- **Offset 0x04:** Dirección del `Reset_Handler`.
- **Offset 0x08 a 0x3C:** Excepciones del núcleo Cortex (NMI, HardFault, SysTick, etc.).
- **Offset 0x40 en adelante:** Interrupciones de periféricos propios del fabricante (USART, TIM, DMA, etc.).

```

1  .section .isr_vector,"a",%progbits
2  .type g_pfnVectors, %object
3
4  g_pfnVectors:
5  .word _estack                /* 0x00: Pila inicial */
6  .word Reset_Handler         /* 0x04: Reset Handler */
7  .word NMI_Handler           /* 0x08: NMI Handler */
8  .word HardFault_Handler     /* 0x0C: HardFault Handler */
9  /* ... Excepciones del nucleo ... */
10 .word SysTick_Handler       /* 0x3C: Timer del sistema */
11
12 /* Interrupciones de perifericos externos */
13 .word WWDG_IRQHandler
14 .word USART1_IRQHandler
15 .word EXTI0_IRQHandler

```

Podemos observar que esta sección (`.isr_vector`) es la que colocaba anteriormente el *linker script* al principio exacto de la **FLASH**, para así ubicar todos estos símbolos en las posiciones adecuadas.

1.3.1. El modificador *weak*

En el archivo de arranque, todas las interrupciones se enlazan por defecto a una rutina genérica llamada `Default_Handler` (que ejecuta un bucle infinito) mediante el atributo de ensamblador `.weak` (o `__attribute__((weak))` en C).

El modificador *weak* (débil) indica al enlazador que use esa definición por defecto únicamente si el usuario no ha definido una función con el mismo nombre en su código C.

```

1  .weak USART1_IRQHandler
2  .thumb_set USART1_IRQHandler, Default_Handler

```

Si el desarrollador escribe en su archivo `main.c`:

```

1  void USART1_IRQHandler(void) {
2      //Codigo propio para procesar la recepcion de datos
3  }

```

El enlazador sustituye automáticamente la dirección del `Default_Handler` por la dirección de esta nueva función dentro de la tabla de vectores, sin necesidad de modificar el archivo de ensamblador original.

1.3.2. Interrupciones y compartición de memoria

Cuando salta una interrupción, el hardware del procesador suspende la ejecución del hilo principal (`main`), guarda automáticamente el contexto (registros R0-R3, R12, LR, PC, xPSR) en la pila y salta a la ISR asignada en la tabla de vectores.

Al compartir variables entre el hilo principal y una rutina de interrupción, surgen problemas de inconsistencia de datos y condiciones de carrera:

- **Uso obligatorio de `volatile`:** Evita que el compilador almacene en registros de la CPU el valor de una variable que puede ser alterada por la ISR en cualquier momento.
- **Operaciones no atómicas:** Si el hilo principal modifica una variable de 32 bits mientras ocurre una interrupción a mitad del proceso de escritura, la ISR leerá datos corruptos.

```
1  #include <stdint.h>
2
3  volatile uint32_t contador_eventos = 0;
4
5  void TIM2_IRQHandler(void) {
6      // esta función se ejecuta automáticamente cuando hay interrupción
7      contador_eventos++;
8  }
9
10 uint32_t leer_contador_seguro(void) {
11     uint32_t copia;
12
13     // Desactivar interrupciones antes de leer
14     __asm volatile ("cpsid i" : : : "memory");
15     copia = contador_eventos; // Lectura protegida
16     // Reactiva interrupciones tras leer
17     __asm volatile ("cpsie i" : : : "memory");
18
19     return copia;
20 }
```